📘 SQL CLASS NOTE – MODULE 2: `FROM`

1. Introduction

The **`FROM` clause** tells SQL *where to look for the data*.
It specifies the **table (or tables)** that the data should come from.

- Think of `FROM` like the *location* in a library:
  - `SELECT` = *what you want* (title, author, page).
  - `FROM` = *where to find it* (which shelf or section).

2. Syntax

```
SELECT column1, column2, ...
FROM table_name;
```

- `table_name` → the table where the columns exist.
- You can use one table or multiple tables (with `JOIN`).

3. Practical Examples with `SchoolDB`

Example 1: Selecting from the **Students** table

```
SELECT FirstName, LastName
FROM Students;
```

- Retrieves student names from the **Students** table.

Example 2: Selecting from the **Courses** table

```
SELECT CourseName, Teacher
FROM Courses;
```

- Retrieves course names and their teachers from **Courses**.

Example 3: Selecting from the **Enrollments** table

```
SELECT StudentID, CourseID, EnrollmentDate
FROM Enrollments;
```

- Shows student-course enrollment details from **Enrollments**.

Example 4: Selecting from multiple tables (Preview of JOIN)

```
SELECT Students.FirstName, Students.LastName, Courses.CourseName
FROM Students, Courses;
```

⚠️ This will produce a **CARTESIAN PRODUCT** (every student matched with every course).

- Not very useful yet, but it introduces the idea that `FROM` can involve multiple tables.
- Later we'll use `JOIN` to link them properly.

Example 5: Using Aliases in `FROM`

```
SELECT s.FirstName, s.LastName, e.EnrollmentDate
FROM Students s, Enrollments e
WHERE s.StudentID = e.StudentID;
```

- `s` is an alias for **Students**, and `e` is an alias for **Enrollments**.
- Makes the query **shorter and cleaner**.

4. Why is `FROM` Important?

- It tells SQL **where to fetch data from**.
- Without `FROM`, SQL doesn't know *which table* contains the data.
- `FROM` is the **bridge** between the question (`SELECT`) and the answer (table data).

5. Real-Life Use Cases

- In a **School system**: Get student names from the **Students** table.
- In a **Banking app**: Get transaction records from the **Transactions** table.
- In a **Hospital**: Fetch patient records from the **Patients** table.
- In **E-commerce**: Retrieve product details from the **Products** table.

6. Classwork (Practice) ✅

👉 Try these queries on **SchoolDB**

1. Select all student names (`FirstName`, `LastName`) from **Students**.
2. Select all course names and codes from **Courses**.
3. Retrieve all `StudentID`, `CourseID`, and `EnrollmentDate` from **Enrollments**.
4. Use table aliases to select `FirstName`, `LastName` from **Students** (alias `s`).
5. Select all data from **Students** but rename the table with alias `st`.

7. Challenge Question 💡

👉 Write a query that selects:

- `FirstName`, `LastName` from the **Students** table (alias = `s`)
- `CourseName` from the **Courses** table (alias = `c`)
- and combine them into one output (⚠️ expect a **Cartesian Product** for now, until we cover `JOIN`).

✅ That's the **expanded Module 2: `FROM`**.

Great 👌 Let's solve **Module 2 Classwork + Challenge** step by step with real SQL queries from our `SchoolDB`.

✅ Solutions to Module 2: `FROM`

1. Select all student names (`FirstName`, `LastName`) from **Students**.

```sql
SELECT FirstName, LastName
FROM Students;
```

📌 Pulls names directly from the **Students** table.

2. Select all course names and codes from **Courses**.

```sql
SELECT CourseName, CourseCode
FROM Courses;
```

📌 Retrieves **subject names** with their **codes**.

3. Retrieve all `StudentID`, `CourseID`, and `EnrollmentDate` from **Enrollments**.

```sql
SELECT StudentID, CourseID, EnrollmentDate
FROM Enrollments;
```

📌 Displays which student enrolled in which course and on what date.

4. Use table aliases to select `FirstName, LastName` from **Students** (alias `s`).

```
SELECT s.FirstName, s.LastName
FROM Students s;
```

📌 Aliases (`s`) make queries **shorter** and **cleaner**.

5. Select all data from **Students** but rename the table with alias `st`.

```
SELECT st.*
FROM Students st;
```

📌 `st.*` selects all columns using the alias.

💡 Challenge Question

👉 Write a query that selects:

- `FirstName`, `LastName` from **Students** (alias `s`)
- `CourseName` from **Courses** (alias `c`)
- Combined into one output (⚠️ Cartesian Product).

```
SELECT s.FirstName, s.LastName, c.CourseName
FROM Students s, Courses c;
```

📌 Result:

- Every student will be paired with every course.
- Example: If there are **200 students** and **8 courses**, this produces **1600 rows**.
- This is why later we'll learn `JOIN` to make these combinations meaningful.

✅ Now you fully understand `FROM`.

👉 Do you want me to move on to **Module 3: `WHERE`** with the same detailed teaching style?